

link github: <https://github.com/caetano3009/Projeto-de-Banco-de-Dados---SisGESC>

```
// =====
// SISTEMA SisGESC - ERP EDUCACIONAL
// Arquivo: sisgesc.dbml
// Para visualizar: cole em https://dbdiagram.io
// =====

Project SisGESC {
  database_type: 'MySQL'
  Note: 'ERP Educacional — Módulos: Base/RH, Acadêmico, Financeiro e BI (Modelo Estrela)'
}

// =====
// MÓDULO 1: BASE E RH
// =====

Table tb_pessoas {
  pk_pessoa_id int [pk, increment, note: 'Identificador único da pessoa']
  nome varchar(100) [not null, note: 'Primeiro nome']
  sobrenome varchar(150) [not null, note: 'Sobrenome completo']
  cpf varchar(14) [unique, not null, note: 'CPF formatado (000.000.000-00)']
  data_nascimento date [not null, note: 'Data de nascimento']
  email_pessoal varchar(255) [unique, note: 'E-mail pessoal']
  telefone varchar(20) [note: 'Telefone com DDD']
  cep char(9) [note: 'CEP formatado (00000-000)']
  logradouro varchar(255) [note: 'Rua ou Avenida']
  numero varchar(10) [note: 'Número do endereço']
  bairro varchar(100) [note: 'Bairro']
  cidade varchar(100) [note: 'Cidade']
  estado char(2) [note: 'UF (ex: SP)']
  criado_em timestamp [default: `CURRENT_TIMESTAMP`, note: 'Auditoria: data de criação']
}

Table tb_funcionarios {
  pk_pessoa_id int [pk, ref: - tb_pessoas.pk_pessoa_id, note: 'FK para tb_pessoas']
  cargo varchar(50) [not null, note: 'Cargo do funcionário']
  salario decimal(10,2) [not null, note: 'Salário mensal bruto']
  fk_gestor_id int [ref: > tb_funcionarios.pk_pessoa_id, note: 'AUTO-RELACIONAMENTO: gestor direto']

  Note: 'Auto-relacionamento: um funcionário pode ser gestor de outros'
}

Table tb_alunos {
```

```

    pk_pessoa_id      int      [pk, ref: - tb_pessoas.pk_pessoa_id, note: 'FK para
tb_pessoas']
    ra                varchar(20) [unique, not null, note: 'Registro Acadêmico (RA) único']
    email_institucional varchar(255) [unique, note: 'E-mail institucional do aluno']
    curso              varchar(100) [not null, note: 'Nome do curso matriculado']
    semestre_atual     int      [default: 1, note: 'Semestre em que o aluno está']
    periodo            varchar(20) [note: 'Matutino, Vespertino ou Noturno']
    modalidade         varchar(20) [note: 'Presencial ou EAD']
    status_matricula   varchar(20) [default: 'Ativo', note: 'Ativo, Trancado, Concluído']
    media_geral        decimal(4,2) [note: 'Média geral calculada']
    carga_horaria_cumprida int    [note: 'Horas cumpridas até o momento']
    carga_horaria_total int      [note: 'Total de horas do curso']
    data_prevista_conclusao date  [note: 'Previsão de formatura']
    data_matricula     date      [default: `CURRENT_DATE`, note: 'Data da matrícula
inicial']
}

```

```

Table tb_professores {
    pk_pessoa_id      int      [pk, ref: - tb_pessoas.pk_pessoa_id, note: 'FK para
tb_pessoas']
    numero_matricula   varchar(20) [unique, not null, note: 'Matrícula funcional do
professor']
    email_institucional varchar(255) [unique, note: 'E-mail institucional']
    titulacao          varchar(50) [note: 'Graduação, Especialização, Mestrado, Doutorado']
    especialidade_principal varchar(100) [note: 'Área de especialidade']
    departamento       varchar(100) [note: 'Departamento vinculado']
    regime_trabalho     varchar(50) [note: '20h, 40h ou Dedicação Exclusiva']
    carga_horaria_semanal int    [note: 'Horas semanais de trabalho']
    salario_base        decimal(10,2) [note: 'Salário base do professor']
    sala               varchar(20) [note: 'Sala ou ramal do professor']
    data_admissao       date      [not null, note: 'Data de contratação']
}

```

```

// =====
// MÓDULO 2: ACADÊMICO
// =====

```

```

Table tb_cursos {
    pk_curso_id int      [pk, increment, note: 'Identificador do curso']
    nome_curso   varchar(100) [not null, note: 'Nome do curso (ex: ADS, Direito)']
    carga_horaria int      [not null, note: 'Carga horária total do curso']
}

```

```

Table tb_turmas {
    pk_turma_id int      [pk, increment, note: 'Identificador da turma']
    fk_curso_id int      [not null, ref: > tb_cursos.pk_curso_id, note: 'FK para o curso']
    nome_turma   varchar(50) [not null, note: 'Nome ou código da turma']
    ano          int      [not null, note: 'Ano letivo da turma']
}

```

}

```
Table tb_disciplinas {  
  pk_disciplina_id int [pk, increment, note: 'Identificador da disciplina']  
  nome_disciplina varchar(100) [not null, note: 'Nome da disciplina']  
  carga_horaria int [not null, note: 'Carga horária da disciplina']  
}
```

```
Table tb_turma_disciplina {  
  pk_turma_disciplina_id int [pk, increment, note: 'Identificador da associação']  
  fk_turma_id int [not null, ref: > tb_turmas.pk_turma_id, note: 'FK para turma']  
  fk_disciplina_id int [not null, ref: > tb_disciplinas.pk_disciplina_id, note: 'FK para disciplina']  
  fk_professor_id int [not null, ref: > tb_professores.pk_pessoa_id, note: 'Professor responsável']  
}
```

Note: 'Entidade associativa N:N — Turma x Disciplina com professor responsável'

}

```
Table tb_matriculas {  
  pk_matricula_id int [pk, increment, note: 'Identificador da matrícula']  
  fk_aluno_id int [not null, ref: > tb_alunos.pk_pessoa_id, note: 'FK para aluno']  
  fk_turma_id int [not null, ref: > tb_turmas.pk_turma_id, note: 'FK para turma']  
  data_matricula date [default: `CURRENT_DATE`, note: 'Data em que a matrícula foi feita']  
}
```

Note: 'Entidade associativa N:N — Aluno x Turma'

}

```
Table tb_notas {  
  pk_nota_id int [pk, increment, note: 'Identificador da nota']  
  fk_matricula_id int [not null, ref: > tb_matriculas.pk_matricula_id, note: 'FK para matrícula']  
  fk_turma_disciplina_id int [not null, ref: > tb_turma_disciplina.pk_turma_disciplina_id, note: 'FK para turma-disciplina']  
  data_avaliacao date [not null, note: 'Data da avaliação']  
  valor_nota decimal(4,2) [not null, note: 'Nota de 0.00 a 10.00 (CHECK constraint no SQL)']  
}
```

```
Table tb_frequencia {  
  pk_frequencia_id int [pk, increment, note: 'Identificador do registro de frequência']  
  fk_matricula_id int [not null, ref: > tb_matriculas.pk_matricula_id, note: 'FK para matrícula']  
  fk_turma_disciplina_id int [not null, ref: > tb_turma_disciplina.pk_turma_disciplina_id, note: 'FK para turma-disciplina']  
  data_aula date [not null, note: 'Data da aula registrada']  
  presenca tinyint(1) [not null, note: '1 = Presente | 0 = Ausente']  
}
```

```
// =====  
// MÓDULO 3: FINANCEIRO  
// =====
```

```
Table tb_contratos {  
  pk_contrato_id int          [pk, increment, note: 'Identificador do contrato']  
  fk_aluno_id   int           [unique, not null, ref: - tb_alunos.pk_pessoa_id, note: 'FK única: 1  
contrato por aluno']  
  valor_total   decimal(10,2) [not null, note: 'Valor total do contrato educacional']  
}
```

```
Table tb_metodos_pagamento {  
  pk_metodo_id int          [pk, increment, note: 'Identificador do método']  
  descricao    varchar(50) [unique, not null, note: 'Ex: PIX, Boletão, Cartão de Crédito']  
}
```

```
Table tb_mensalidades {  
  pk_mensalidade_id int      [pk, increment, note: 'Identificador da mensalidade']  
  fk_contrato_id   int       [not null, ref: > tb_contratos.pk_contrato_id, note: 'FK para  
contrato']  
  valor_mensalidade decimal(10,2) [not null, note: 'Valor da parcela mensal']  
  data_vencimento  date       [not null, note: 'Data limite para pagamento']  
  status_mensalidade varchar(20) [default: 'Pendente', note: 'Pendente, Pago ou Atrasado']  
}
```

```
Table tb_pagamentos {  
  pk_pagamento_id int      [pk, increment, note: 'Identificador do pagamento']  
  fk_mensalidade_id int     [unique, not null, ref: - tb_mensalidades.pk_mensalidade_id,  
note: 'FK única: 1 pagamento por mensalidade']  
  fk_metodo_id   int        [not null, ref: > tb_metodos_pagamento.pk_metodo_id, note: 'FK  
para método de pagamento']  
  data_pagamento date       [not null, note: 'Data em que o pagamento foi efetuado']  
  valor_pago     decimal(10,2) [not null, note: 'Valor efetivamente pago']  
}
```

```
Table tb_historico_descontos {  
  pk_desconto_id int      [pk, increment, note: 'Identificador do desconto']  
  fk_contrato_id int      [not null, ref: > tb_contratos.pk_contrato_id, note: 'FK para  
contrato']  
  valor_desconto decimal(10,2) [not null, note: 'Valor do desconto concedido']  
  motivo          varchar(100) [not null, note: 'Motivo do desconto (ex: bolsa, convênio)']  
  data_concessao  date         [not null, note: 'Data em que o desconto foi concedido']  
  data_expiracao  date         [note: 'Data de expiração (NULL = sem prazo)']  
  status_ativo    boolean      [default: true, note: 'TRUE = desconto ativo']  
}
```

```
Table tb_acordos_renegociacao {
```

```

    pk_acordo_id      int      [pk, increment, note: 'Identificador do acordo']
    fk_contrato_id     int      [not null, ref: > tb_contratos.pk_contrato_id, note: 'FK para
contrato']
    data_acordo        date      [default: `CURRENT_DATE`, note: 'Data do acordo
firmado']
    valor_total_renegociado decimal(10,2) [not null, note: 'Valor total a ser pago no acordo']
    quantidade_parcelas int      [not null, note: 'Número de parcelas do acordo']
    status_acordo       varchar(20) [default: 'Ativo', note: 'Ativo, Quitado ou Quebrado']
}

```

```

Table tb_ocorrencias_cobranca {
    pk_ocorrencia_id int      [pk, increment, note: 'Identificador da ocorrência']
    fk_aluno_id      int      [not null, ref: > tb_alunos.pk_pessoa_id, note: 'FK para aluno
inadimplente']
    data_contato     datetime [default: `CURRENT_TIMESTAMP`, note: 'Data e hora do
contato']
    meio_contato     varchar(20) [not null, note: 'WhatsApp, E-mail, Telefone, Presencial']
    status_contato   varchar(50) [not null, note: 'Contatado, Sem Retorno, Prometeu Pagar']
    resumo_interacao text      [note: 'Resumo livre da conversa']
}

```

```

// =====
// MÓDULO 4: DATA WAREHOUSE / BI (Modelo Estrela)
// =====

```

```

Table tb_dim_tempo {
    pk_tempo_id      int [pk, increment, note: 'Surrogate Key da dimensão tempo']
    data_referencia date [not null, note: 'Data completa de referência']
    ano              int [not null, note: 'Ano (ex: 2026)']
    mes              int [not null, note: 'Mês (1 a 12)']
    trimestre        int [not null, note: 'Trimestre (1 a 4)']
    semestre         int [not null, note: 'Semestre (1 ou 2)']

```

```

indexes {
    (ano, mes) [unique]
}

```

```

Note: 'Dimensão Tempo — granularidade mensal'
}

```

```

Table tb_dim_aluno {
    sk_aluno_id      int      [pk, increment, note: 'Surrogate Key do aluno (OLAP)']
    fk_aluno_oltp    int      [not null, note: 'Referência ao pk_pessoa_id do OLTP']
    nome_completo    varchar(255) [not null, note: 'Nome completo concatenado para análise']
    ra               varchar(20) [not null, note: 'Registro Acadêmico']
    curso            varchar(100) [not null, note: 'Nome do curso']
    modalidade       varchar(20) [note: 'Presencial ou EAD']
    periodo          varchar(20) [note: 'Matutino, Vespertino ou Noturno']

```

Note: 'Dimensão Aluno — separada do OLTP para o modelo estrela'

}

Table tb_dim_curso {
 sk_curso_id int [pk, increment, note: 'Surrogate Key do curso (OLAP)']
 fk_curso_oltp int [not null, note: 'Referência ao pk_curso_id do OLTP']
 nome_curso varchar(100) [not null, note: 'Nome do curso']
 carga_horaria int [not null, note: 'Carga horária total']

Note: 'Dimensão Curso — separada do OLTP para o modelo estrela'

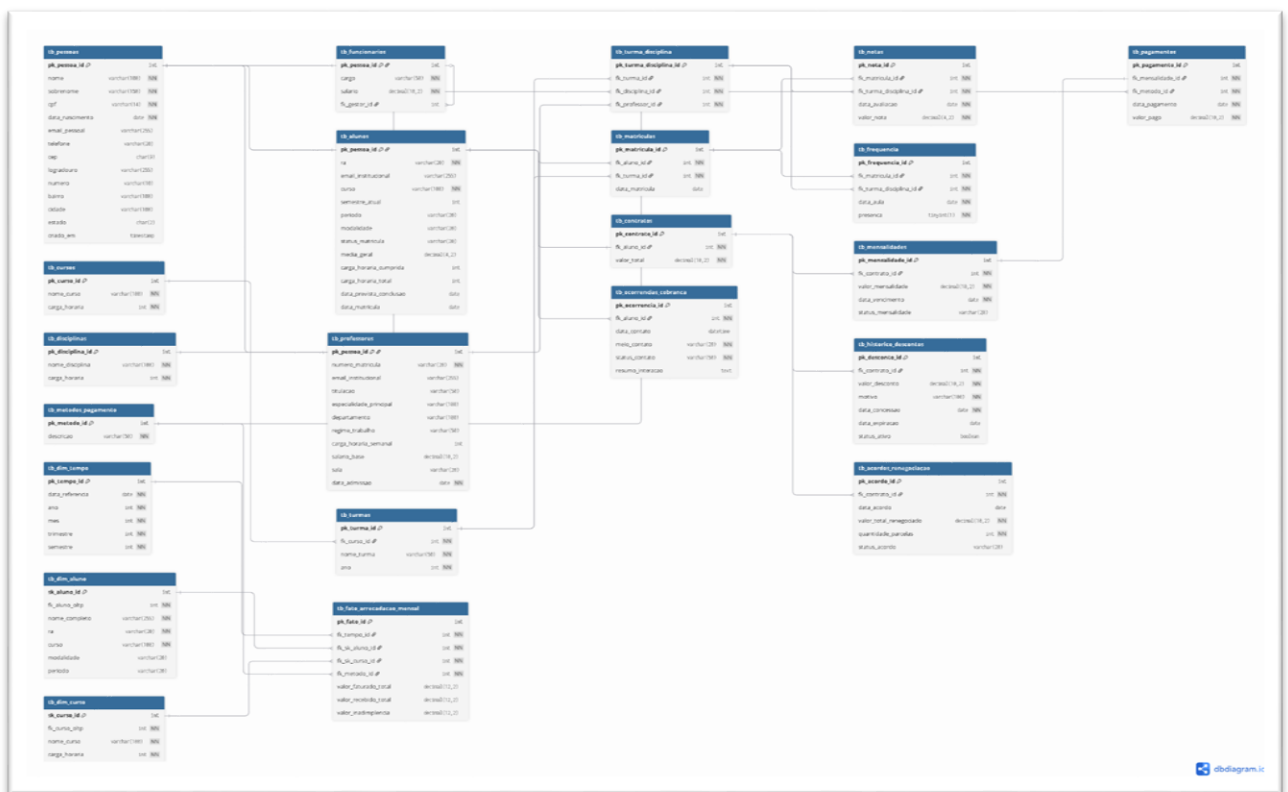
}

Table tb_fato_arrecadacao_mensal {
 pk_fato_id int [pk, increment, note: 'Surrogate Key da tabela fato']
 fk_tempo_id int [not null, ref: > tb_dim_tempo.pk_tempo_id, note: 'FK para dimensão tempo']
 fk_sk_aluno_id int [not null, ref: > tb_dim_aluno.sk_aluno_id, note: 'FK para dimensão aluno']
 fk_sk_curso_id int [not null, ref: > tb_dim_curso.sk_curso_id, note: 'FK para dimensão curso']
 fk_metodo_id int [not null, ref: > tb_metodos_pagamento.pk_metodo_id, note: 'FK para método de pagamento']
 valor_faturado_total decimal(12,2) [note: 'Total faturado no mês']
 valor_recebido_total decimal(12,2) [note: 'Total efetivamente recebido']
 valor_inadimplencia decimal(12,2) [note: 'Total em atraso no mês']

Note: 'Tabela Fato — granularidade: 1 linha = 1 pagamento por aluno por mês'

}

-Diagrama



Dicionário de Dados Consolidado

MÓDULO 1 – BASE E RH

1. tb_pessoas

Descrição: Tabela base para centralizar informações de todas as pessoas (alunos, funcionários, professores) no sistema.

Coluna	Tipo de Dados	Restrições	Descrição
pk_pessoa_id	INT	PRIMARY KEY, AUTO_INCREMENT	Identificador único da pessoa.
nome	VARCHAR (100)	NOT NULL	Primeiro nome.
sobrenome	VARCHAR (150)	NOT NULL	Sobrenome completo.
cpf	VARCHAR (14)	UNIQUE, NOT NULL	CPF formatado (000.000.000-00).
data_nascimento	DATE	NOT NULL	Data de nascimento.
email_pessoal	VARCHAR (255)	UNIQUE	E-mail pessoal.
telefone	VARCHAR (20)		Telefone com DDD.
cep	CHAR (9)		CEP formatado (00000-000).
logradouro	VARCHAR (255)		Rua ou Avenida.
numero	VARCHAR (10)		Número do endereço.
bairro	VARCHAR (100)		Bairro.
cidade	VARCHAR (100)		Cidade.
estado	CHAR (2)		UF (ex: SP).
criado_em	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Auditoria: data de criação.

2. tb_funcionarios

Descrição: Informações específicas de funcionários, herdando dados de tb_pessoas. Possui auto-relacionamento para gestor.

Coluna	Tipo de Dados	Restrições	Descrição
pk_pessoa_id	INT	PRIMARY KEY, FOREIGN KEY (tb_pessoas)	FK para tb_pessoas.
cargo	VARCHAR (50)	NOT NULL	Cargo do funcionário.
salario	DECIMAL (10,2)	NOT NULL	Salário mensal bruto.
fk_gestor_id	INT	FOREIGN KEY (tb_funcionarios)	Auto-relacionamento: gestor direto.

3. tb_alunos

Descrição: Informações específicas de alunos, herdando dados de tb_pessoas.

Coluna	Tipo de Dados	Restrições	Descrição
pk_pessoa_id	INT	PRIMARY KEY, FOREIGN KEY (tb_pessoas)	FK para tb_pessoas.
ra	VARCHAR (20)	UNIQUE, NOT NULL	Registro Acadêmico (RA) único.
email_institucional	VARCHAR (255)	UNIQUE	E-mail institucional do aluno.
curso	VARCHAR (100)	NOT NULL	Nome do curso matriculado.
semestre_atual	INT	DEFAULT 1	Semestre em que o aluno está.
periodo	VARCHAR (20)		Matutino, Vespertino ou Noturno.
modalidade	VARCHAR (20)		Presencial ou EAD.
status_matricula	VARCHAR (20)	DEFAULT 'Ativo'	Ativo, Trancado, Concluído.
media_geral	DECIMAL (4,2)		Média geral calculada.
carga_horaria_cumprida	INT		Horas cumpridas até o momento.
carga_horaria_total	INT		Total de horas do curso.
data_prevista_conclusao	DATE		Previsão de formatura.
data_matricula	DATE	DEFAULT CURRENT_DATE	Data da matrícula inicial.

4. tb_professores

Descrição: Informações específicas de professores, herdando dados de tb_pessoas.

Coluna	Tipo de Dados	Restrições	Descrição
pk_pessoa_id	INT	PRIMARY KEY, FOREIGN KEY (tb_pessoas)	FK para tb_pessoas.
numero_matricula	VARCHAR(20)	UNIQUE, NOT NULL	Matrícula funcional do professor.
email_institucional	VARCHAR(255)	UNIQUE	E-mail institucional.
titulacao	VARCHAR(50)		Graduação, Especialização, Mestrado, Doutorado.
especialidade_principal	VARCHAR(100)		Área de especialidade.
departamento	VARCHAR(100)		Departamento vinculado.
regime_trabalho	VARCHAR(50)		20h, 40h ou Dedicação Exclusiva.
carga_horaria_semanal	INT		Horas semanais de trabalho.
salario_base	DECIMAL(10,2)		Salário base do professor.
sala	VARCHAR(20)		Sala ou ramal do professor.
data_admissao	DATE	NOT NULL	Data de contratação.

MÓDULO 2 – ACADÊMICO

5. tb_cursos

Descrição: Armazena os cursos acadêmicos oferecidos pela instituição.

Coluna	Tipo de Dados	Restrições	Descrição
pk_curso_id	INT	PRIMARY KEY, AUTO_INCREMENT	Identificador do curso.
nome_curso	VARCHAR(100)	NOT NULL	Nome do curso (ex: ADS, Direito).
carga_horaria	INT	NOT NULL	Carga horária total do curso.

6. tb_turmas

Descrição: Organiza os alunos em grupos para um curso específico em um determinado ano.

Coluna	Tipo de Dados	Restrições	Descrição
pk_turma_id	INT	PRIMARY KEY, AUTO_INCREMENT	Identificador da turma.

Coluna	Tipo de Dados	Restrições	Descrição
fk_curso_id	INT	NOT NULL, FOREIGN KEY (tb_cursos)	FK para o curso.
nome_turma	VARCHAR(50)	NOT NULL	Nome ou código da turma.
ano	INT	NOT NULL	Ano letivo da turma.

7. tb_disciplinas

Descrição: Armazena as disciplinas oferecidas pelos cursos.

Coluna	Tipo de Dados	Restrições	Descrição
pk_disciplina_id	INT	PRIMARY KEY, AUTO_INCREMENT	Identificador da disciplina.
nome_disciplina	VARCHAR(100)	NOT NULL	Nome da disciplina.
carga_horaria	INT	NOT NULL	Carga horária da disciplina.

8. tb_turma_disciplina

Descrição: Entidade associativa N:N entre Turma e Disciplina com o professor responsável.

Coluna	Tipo de Dados	Restrições	Descrição
pk_turma_disciplina_id	INT	PRIMARY KEY, AUTO_INCREMENT	Identificador da associação.
fk_turma_id	INT	NOT NULL, FOREIGN KEY (tb_turmas)	FK para turma.
fk_disciplina_id	INT	NOT NULL, FOREIGN KEY (tb_disciplinas)	FK para disciplina.
fk_professor_id	INT	NOT NULL, FOREIGN KEY (tb_professores)	Professor responsável pela turma-disciplina.

9. tb_matriculas

Descrição: Entidade associativa N:N entre Aluno e Turma. Registra a matrícula do aluno.

Coluna	Tipo de Dados	Restrições	Descrição
pk_matricula_id	INT	PRIMARY KEY, AUTO_INCREMENT	Identificador da matrícula.
fk_aluno_id	INT	NOT NULL, FOREIGN KEY (tb_alunos)	FK para aluno.
fk_turma_id	INT	NOT NULL, FOREIGN KEY (tb_turmas)	FK para turma.

Coluna	Tipo de Dados	Restrições	Descrição
data_matricula	DATE	DEFAULT CURRENT_DATE	Data em que a matrícula foi feita.

10. tb_notas

Descrição: Registra as notas dos alunos por matrícula e turma-disciplina.

Coluna	Tipo de Dados	Restrições	Descrição
pk_nota_id	INT	PRIMARY KEY, AUTO_INCREMENT	Identificador da nota.
fk_matricula_id	INT	NOT NULL, FOREIGN KEY (tb_matriculas)	FK para matrícula.
fk_turma_disciplina_id	INT	NOT NULL, FOREIGN KEY (tb_turma_disciplina)	FK para turma-disciplina.
data_avaliacao	DATE	NOT NULL	Data da avaliação.
valor_nota	DECIMAL(4,2)	NOT NULL, CHECK (0 a 10)	Nota de 0.00 a 10.00.

11. tb_frequencia

Descrição: Registra a frequência diária dos alunos por matrícula e turma-disciplina.

Coluna	Tipo de Dados	Restrições	Descrição
pk_frequencia_id	INT	PRIMARY KEY, AUTO_INCREMENT	Identificador do registro de frequência.
fk_matricula_id	INT	NOT NULL, FOREIGN KEY (tb_matriculas)	FK para matrícula.
fk_turma_disciplina_id	INT	NOT NULL, FOREIGN KEY (tb_turma_disciplina)	FK para turma-disciplina.
data_aula	DATE	NOT NULL	Data da aula registrada.
presenca	TINYINT(1)	NOT NULL	1 = Presente 0 = Ausente.

MÓDULO 3 – FINANCEIRO

12. tb_contratos

Descrição: Registra os contratos financeiros dos alunos. Um contrato por aluno.

Coluna	Tipo de Dados	Restrições	Descrição
pk_contrato_id	INT	PRIMARY KEY, AUTO_INCREMENT	Identificador do contrato.
fk_aluno_id	INT	UNIQUE, NOT NULL, FOREIGN KEY (tb_alunos)	FK única: 1 contrato por aluno.
valor_total	DECIMAL(10,2)	NOT NULL	Valor total do contrato educacional.

13. tb_metodos_pagamento

Descrição: Métodos de pagamento aceitos pela instituição.

Coluna	Tipo de Dados	Restrições	Descrição
pk_metodo_id	INT	PRIMARY KEY, AUTO_INCREMENT	Identificador do método.
descricao	VARCHAR(50)	UNIQUE, NOT NULL	Ex: PIX, Boletto, Cartão de Crédito.

14. tb_mensalidades

Descrição: Detalhes das mensalidades a serem pagas pelos alunos.

Coluna	Tipo de Dados	Restrições	Descrição
pk_mensalidade_id	INT	PRIMARY KEY, AUTO_INCREMENT	Identificador da mensalidade.
fk_contrato_id	INT	NOT NULL, FOREIGN KEY (tb_contratos)	FK para contrato.
valor_mensalidade	DECIMAL(10,2)	NOT NULL	Valor da parcela mensal.
data_vencimento	DATE	NOT NULL	Data limite para pagamento.
status_mensalidade	VARCHAR(20)	DEFAULT 'Pendente'	Pendente, Pago ou Atrasado.

15. tb_pagamentos

Descrição: Registra os pagamentos efetuados pelos alunos. Um pagamento por mensalidade.

Coluna	Tipo de Dados	Restrições	Descrição
pk_pagamento_id	INT	PRIMARY KEY, AUTO_INCREMENT	Identificador do pagamento.
fk_mensalidade_id	INT	UNIQUE, NOT NULL, FOREIGN KEY (tb_mensalidades)	FK única: 1 pagamento por mensalidade.

Coluna	Tipo de Dados	Restrições	Descrição
fk_metodo_id	INT	NOT NULL, FOREIGN KEY (tb_metodos_pagamento)	FK para método de pagamento.
data_pagamento	DATE	NOT NULL	Data em que o pagamento foi efetuado.
valor_pago	DECIMAL(10,2)	NOT NULL	Valor efetivamente pago.

16. tb_historico_descontos

Descrição: Registra o histórico de descontos concedidos nos contratos dos alunos.

Coluna	Tipo de Dados	Restrições	Descrição
pk_desconto_id	INT	PRIMARY KEY, AUTO_INCREMENT	Identificador do desconto.
fk_contrato_id	INT	NOT NULL, FOREIGN KEY (tb_contratos)	FK para contrato.
valor_desconto	DECIMAL(10,2)	NOT NULL	Valor do desconto concedido.
motivo	VARCHAR(100)	NOT NULL	Motivo do desconto (ex: bolsa, convênio).
data_concessao	DATE	NOT NULL	Data em que o desconto foi concedido.
data_expiracao	DATE		Data de expiração (NULL = sem prazo).
status_ativo	BOOLEAN	DEFAULT TRUE	TRUE = desconto ativo.

17. tb_acordos_renegociacao

Descrição: Registra acordos de renegociação de dívidas dos contratos.

Coluna	Tipo de Dados	Restrições	Descrição
pk_acordo_id	INT	PRIMARY KEY, AUTO_INCREMENT	Identificador do acordo.
fk_contrato_id	INT	NOT NULL, FOREIGN KEY (tb_contratos)	FK para contrato.
data_acordo	DATE	DEFAULT CURRENT_DATE	Data do acordo firmado.
valor_total_renegociado	DECIMAL(10,2)	NOT NULL	Valor total a ser pago no acordo.
quantidade_parcelas	INT	NOT NULL	Número de parcelas do acordo.
status_acordo	VARCHAR(20)	DEFAULT 'Ativo'	Ativo, Quitado ou Quebrado.

18. tb_ocorrencias_cobranca

Descrição: Registra as tentativas de contato com alunos inadimplentes.

Coluna	Tipo de Dados	Restrições	Descrição
pk_ocorrencia_id	INT	PRIMARY KEY, AUTO_INCREMENT	Identificador da ocorrência.
fk_aluno_id	INT	NOT NULL, FOREIGN KEY (tb_alunos)	FK para aluno inadimplente.
data_contato	DATETIME	DEFAULT CURRENT_TIMESTAMP	Data e hora do contato.
meio_contato	VARCHAR(20)	NOT NULL	WhatsApp, E-mail, Telefone, Presencial.
status_contato	VARCHAR(50)	NOT NULL	Contatado, Sem Retorno, Prometeu Pagar.
resumo_interacao	TEXT		Resumo livre da conversa.

MÓDULO 4 – DATA WAREHOUSE / BI (Modelo Estrela)

19. tb_dim_tempo

Descrição: Dimensão de tempo para o Data Warehouse. Granularidade mensal. Usada em todas as tabelas fato.

Coluna	Tipo de Dados	Restrições	Descrição
pk_tempo_id	INT	PRIMARY KEY, AUTO_INCREMENT	Surrogate Key da dimensão tempo.
data_referencia	DATE	NOT NULL	Data completa de referência.
ano	INT	NOT NULL	Ano (ex: 2026).
mes	INT	NOT NULL	Mês (1 a 12).
trimestre	INT	NOT NULL	Trimestre (1 a 4).
semestre	INT	NOT NULL	Semestre (1 ou 2).
UNIQUE INDEX	—	UNIQUE (ano, mes)	Garante um registro único por mês/ano.

20. tb_dim_aluno

Descrição: Dimensão analítica do aluno para o DW. Usa Surrogate Key separada do OLTP.

Coluna	Tipo de Dados	Restrições	Descrição
sk_aluno_id	INT	PRIMARY KEY, AUTO_INCREMENT	Surrogate Key do aluno (OLAP).
fk_aluno_oltp	INT	NOT NULL	Referência ao pk_pessoa_id do OLTP.
nome_completo	VARCHAR(255)	NOT NULL	Nome completo concatenado para análise.
ra	VARCHAR(20)	NOT NULL	Registro Acadêmico.
curso	VARCHAR(100)	NOT NULL	Nome do curso.
modalidade	VARCHAR(20)		Presencial ou EAD.
periodo	VARCHAR(20)		Matutino, Vespertino ou Noturno.

21. tb_dim_curso

Descrição: Dimensão analítica do curso para o DW. Usa Surrogate Key separada do OLTP.

Coluna	Tipo de Dados	Restrições	Descrição
sk_curso_id	INT	PRIMARY KEY, AUTO_INCREMENT	Surrogate Key do curso (OLAP).
fk_curso_oltp	INT	NOT NULL	Referência ao pk_curso_id do OLTP.
nome_curso	VARCHAR(100)	NOT NULL	Nome do curso.
carga_horaria	INT	NOT NULL	Carga horária total.

22. tb_fato_arrecadacao_mensal

Descrição: Tabela fato para análise financeira mensal. Granularidade: 1 linha = 1 pagamento por aluno por mês.

Coluna	Tipo de Dados	Restrições	Descrição
pk_fato_id	INT	PRIMARY KEY, AUTO_INCREMENT	Surrogate Key da tabela fato.
fk_tempo_id	INT	NOT NULL, FOREIGN KEY (tb_dim_tempo)	FK para dimensão tempo.
fk_sk_aluno_id	INT	NOT NULL, FOREIGN KEY (tb_dim_aluno)	FK para dimensão aluno.
fk_sk_curso_id	INT	NOT NULL, FOREIGN KEY (tb_dim_curso)	FK para dimensão curso.
fk_metodo_id	INT	NOT NULL, FOREIGN KEY (tb_metodos_pagamento)	FK para método de pagamento.

Coluna	Tipo de Dados	Restrições	Descrição
valor_faturado_total	DECIMAL(12,2)		Total faturado no mês.
valor_recebido_total	DECIMAL(12,2)		Total efetivamente recebido.
valor_inadimplencia	DECIMAL(12,2)		Total em atraso no mês.

- Carga De Dados

```
-- =====  
-- SCRIPT UNIFICADO: SISTEMA SisGESC - ERP EDUCACIONAL E RH  
-- OBJETIVO: Instalação, Carga de Dados e Validação de Integridade  
-- =====
```

```
-- -----  
-- FASE 0: RESET DO AMBIENTE  
-- -----
```

```
DROP DATABASE IF EXISTS db_sisgesc;  
CREATE DATABASE db_sisgesc;  
USE db_sisgesc;
```

```
-- -----  
-- FASE 1: DDL - CRIAÇÃO DA ESTRUTURA (MÓDULOS)  
-- -----
```

```
-- MÓDULO 1: BASE (Centralização de Pessoas)  
CREATE TABLE tb_pessoas (  
    pk_pessoa_id INT PRIMARY KEY AUTO_INCREMENT,  
    nome VARCHAR(100) NOT NULL,  
    sobrenome VARCHAR(150) NOT NULL,  
    cpf VARCHAR(14) UNIQUE NOT NULL,  
    data_nascimento DATE NOT NULL,  
    email_pessoal VARCHAR(255) UNIQUE,  
    telefone VARCHAR(20),  
    cep CHAR(9),  
    logradouro VARCHAR(255),  
    numero VARCHAR(10),  
    bairro VARCHAR(100),  
    cidade VARCHAR(100),  
    estado CHAR(2),  
    criado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
) ENGINE=InnoDB;
```

```
-- MÓDULO 2: RECURSOS HUMANOS  
CREATE TABLE tb_departamentos (  
    pk_departamento INT AUTO_INCREMENT PRIMARY KEY,  
    nome_departamento VARCHAR(100) NOT NULL UNIQUE,  
    descricao VARCHAR(200),  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
) ENGINE=InnoDB;
```

```
CREATE TABLE tb_cargos (  
    pk_cargo INT AUTO_INCREMENT PRIMARY KEY,  
    nome_cargo VARCHAR(100) NOT NULL UNIQUE,  
    nivel ENUM('operacional', 'tecnico', 'supervisao', 'gerencia', 'direcao') NOT NULL,  
    salario_base DECIMAL(10,2) NOT NULL,  
    carga_horaria_semanal TINYINT UNSIGNED NOT NULL,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
) ENGINE=InnoDB;
```

```
CREATE TABLE tb_funcionarios (  
    pk_pessoa_id INT PRIMARY KEY,  
    registro_funcionario CHAR(10) NOT NULL UNIQUE,  
    fk_cargo INT NOT NULL,  
    fk_departamento INT NOT NULL,  
    fk_gestor_id INT NULL,  
    regime_trabalho ENUM('clt', 'pj', 'estagio', 'temporario') NOT NULL,  
    status_funcionario ENUM('ativo', 'inativo', 'afastado', 'ferias') DEFAULT 'ativo',
```

```

data_admissao DATE NOT NULL,
data_demissao DATE,
salario_vigente DECIMAL(10,2) NOT NULL,
CONSTRAINT fk_func_pessoa FOREIGN KEY (pk_pessoa_id) REFERENCES tb_pessoas(pk_pessoa_id) ON DELETE
CASCADE,
CONSTRAINT fk_func_cargo FOREIGN KEY (fk_cargo) REFERENCES tb_cargos(pk_cargo),
CONSTRAINT fk_func_depto FOREIGN KEY (fk_departamento) REFERENCES tb_departamentos(pk_departamento),
CONSTRAINT fk_func_gestor FOREIGN KEY (fk_gestor_id) REFERENCES tb_funcionarios(pk_pessoa_id)
) ENGINE=InnoDB;

```

```

CREATE TABLE tb_historico_cargos (
pk_historico INT AUTO_INCREMENT PRIMARY KEY,
fk_funcionario INT NOT NULL,
fk_cargo INT NOT NULL,
fk_departamento INT NOT NULL,
salario_vigente DECIMAL(10,2) NOT NULL,
motivo ENUM('admissao', 'promocao', 'transferencia', 'ajuste_salarial', 'demissao') NOT NULL,
data_inicio DATE NOT NULL,
data_fim DATE,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
FOREIGN KEY (fk_funcionario) REFERENCES tb_funcionarios(pk_pessoa_id),
FOREIGN KEY (fk_cargo) REFERENCES tb_cargos(pk_cargo),
FOREIGN KEY (fk_departamento) REFERENCES tb_departamentos(pk_departamento)
) ENGINE=InnoDB;

```

```

CREATE TABLE tb_folha_pagamento (
pk_folha INT AUTO_INCREMENT PRIMARY KEY,
fk_funcionario INT NOT NULL,
competencia DATE NOT NULL,
salario_bruto DECIMAL(10,2) NOT NULL,
desconto_inss DECIMAL(10,2) NOT NULL DEFAULT 0.00,
desconto_irrf DECIMAL(10,2) NOT NULL DEFAULT 0.00,
outros_descontos DECIMAL(10,2) NOT NULL DEFAULT 0.00,
salario_liquido DECIMAL(10,2) NOT NULL,
status_folha ENUM('pendente', 'processado', 'pago') DEFAULT 'pendente',
data_pagamento DATE,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
CONSTRAINT uk_folha_func_comp UNIQUE (fk_funcionario, competencia),
FOREIGN KEY (fk_funcionario) REFERENCES tb_funcionarios(pk_pessoa_id)
) ENGINE=InnoDB;

```

```

CREATE TABLE tb_afastamentos (
pk_afastamento INT AUTO_INCREMENT PRIMARY KEY,
fk_funcionario INT NOT NULL,
tipo_afastamento ENUM('ferias', 'licenca_medica', 'licenca_maternidade', 'licenca_paternidade', 'suspensao', 'outro')
NOT NULL,
data_inicio DATE NOT NULL,
data_fim DATE,
motivo VARCHAR(200),
FOREIGN KEY (fk_funcionario) REFERENCES tb_funcionarios(pk_pessoa_id)
) ENGINE=InnoDB;

```

-- MÓDULO 3: ACADÊMICO

```

CREATE TABLE tb_alunos (
pk_pessoa_id INT PRIMARY KEY,
ra VARCHAR(20) UNIQUE NOT NULL,
email_institucional VARCHAR(255) UNIQUE,
curso VARCHAR(100) NOT NULL,
semestre_atual INT DEFAULT 1,
status_matricula VARCHAR(20) DEFAULT 'Ativo',
CONSTRAINT fk_aluno_pessoa FOREIGN KEY (pk_pessoa_id) REFERENCES tb_pessoas(pk_pessoa_id) ON
DELETE CASCADE
) ENGINE=InnoDB;

```

```

CREATE TABLE tb_professores (
pk_pessoa_id INT PRIMARY KEY,

```

```

    numero_matricula VARCHAR(20) UNIQUE NOT NULL,
    especialidade_principal VARCHAR(100),
    titulacao VARCHAR(50),
    CONSTRAINT fk_prof_pessoa FOREIGN KEY (pk_pessoa_id) REFERENCES tb_pessoas(pk_pessoa_id) ON DELETE
CASCADE
) ENGINE=InnoDB;

```

```

CREATE TABLE tb_cursos (
    pk_curso_id INT PRIMARY KEY AUTO_INCREMENT,
    nome_curso VARCHAR(100) NOT NULL,
    carga_horaria INT NOT NULL
) ENGINE=InnoDB;

```

```

CREATE TABLE tb_turmas (
    pk_turma_id INT PRIMARY KEY AUTO_INCREMENT,
    fk_curso_id INT NOT NULL,
    nome_turma VARCHAR(50) NOT NULL,
    ano YEAR NOT NULL,
    CONSTRAINT fk_turma_curso FOREIGN KEY (fk_curso_id) REFERENCES tb_cursos(pk_curso_id)
) ENGINE=InnoDB;

```

```

CREATE TABLE tb_matriculas (
    pk_matricula_id INT PRIMARY KEY AUTO_INCREMENT,
    fk_aluno_id INT NOT NULL,
    fk_turma_id INT NOT NULL,
    data_matricula DATE DEFAULT (CURRENT_DATE),
    CONSTRAINT fk_mat_aluno FOREIGN KEY (fk_aluno_id) REFERENCES tb_alunos(pk_pessoa_id),
    CONSTRAINT fk_mat_turma FOREIGN KEY (fk_turma_id) REFERENCES tb_turmas(pk_turma_id)
) ENGINE=InnoDB;

```

-- MÓDULO 4: FINANCEIRO

```

CREATE TABLE tb_contratos (
    pk_contrato_id INT PRIMARY KEY AUTO_INCREMENT,
    fk_aluno_id INT UNIQUE NOT NULL,
    valor_total DECIMAL(10,2) NOT NULL,
    CONSTRAINT fk_contrato_aluno FOREIGN KEY (fk_aluno_id) REFERENCES tb_alunos(pk_pessoa_id)
) ENGINE=InnoDB;

```

```

CREATE TABLE tb_mensalidades (
    pk_mensalidade_id INT PRIMARY KEY AUTO_INCREMENT,
    fk_contrato_id INT NOT NULL,
    valor_mensalidade DECIMAL(10,2) NOT NULL,
    data_vencimento DATE NOT NULL,
    status_mensalidade VARCHAR(20) DEFAULT 'Pendente',
    CONSTRAINT fk_mensalidade_contrato FOREIGN KEY (fk_contrato_id) REFERENCES tb_contratos(pk_contrato_id)
) ENGINE=InnoDB;

```

```

CREATE TABLE tb_metodos_pagamento (
    pk_metodo_id INT PRIMARY KEY AUTO_INCREMENT,
    descricao VARCHAR(50) UNIQUE NOT NULL
) ENGINE=InnoDB;

```

```

CREATE TABLE tb_pagamentos (
    pk_pagamento_id INT PRIMARY KEY AUTO_INCREMENT,
    fk_mensalidade_id INT UNIQUE NOT NULL,
    fk_metodo_id INT NOT NULL,
    data_pagamento DATE NOT NULL,
    valor_pago DECIMAL(10,2) NOT NULL,
    CONSTRAINT fk_pag_mensalidade FOREIGN KEY (fk_mensalidade_id) REFERENCES
tb_mensalidades(pk_mensalidade_id),
    CONSTRAINT fk_pag_metodo FOREIGN KEY (fk_metodo_id) REFERENCES tb_metodos_pagamento(pk_metodo_id)
) ENGINE=InnoDB;

```

-- MÓDULO 5: DATA WAREHOUSE (BI)

```

CREATE TABLE tb_dim_tempo (
    pk_tempo_id INT PRIMARY KEY AUTO_INCREMENT,

```

```

    data_referencia DATE NOT NULL,
    ano INT NOT NULL,
    mes INT NOT NULL,
    UNIQUE INDEX idx_ano_mes (ano, mes)
) ENGINE=InnoDB;

```

```

CREATE TABLE tb_fato_arrecadacao_mensal (
    pk_fato_id INT PRIMARY KEY AUTO_INCREMENT,
    fk_tempo_id INT NOT NULL,
    fk_curso_id INT NOT NULL,
    fk_aluno_id INT NOT NULL,
    fk_metodo_id INT NOT NULL,
    valor_recebido_total DECIMAL(12,2),
    CONSTRAINT fk_fato_tempo FOREIGN KEY (fk_tempo_id) REFERENCES tb_dim_tempo(pk_tempo_id),
    CONSTRAINT fk_fato_curso FOREIGN KEY (fk_curso_id) REFERENCES tb_cursos(pk_curso_id),
    CONSTRAINT fk_fato_aluno FOREIGN KEY (fk_aluno_id) REFERENCES tb_alunos(pk_pessoa_id)
) ENGINE=InnoDB;

```

```

-----
-- FASE 2: TRIGGERS (LÓGICA DE NEGÓCIO)
-----

```

```

DELIMITER $$

```

```

CREATE TRIGGER tr_funcionario_demissao
BEFORE UPDATE ON tb_funcionarios
FOR EACH ROW
BEGIN
    IF NEW.status_funcionario = 'inativo' AND NEW.data_demissao IS NULL THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Informe a data de demissão ao inativar um funcionário.';
    END IF;
END$$

```

```

CREATE TRIGGER tr_historico_cargo_automatico
AFTER UPDATE ON tb_funcionarios
FOR EACH ROW
BEGIN
    IF OLD.fk_cargo != NEW.fk_cargo OR OLD.fk_departamento != NEW.fk_departamento OR OLD.salario_vigente !=
NEW.salario_vigente THEN
        UPDATE tb_historico_cargos SET data_fim = CURRENT_DATE WHERE fk_funcionario = OLD.pk_pessoa_id AND
data_fim IS NULL;
        INSERT INTO tb_historico_cargos (fk_funcionario, fk_cargo, fk_departamento, salario_vigente, motivo,
data_inicio)
        VALUES (NEW.pk_pessoa_id, NEW.fk_cargo, NEW.fk_departamento, NEW.salario_vigente, 'promocao',
CURRENT_DATE);
    END IF;
END$$

```

```

CREATE TRIGGER tr_calcular_salario_liquido
BEFORE INSERT ON tb_folha_pagamento
FOR EACH ROW
BEGIN
    SET NEW.salario_liquido = NEW.salario_bruto - NEW.desconto_inss - NEW.desconto_irrf - NEW.outros_descontos;
END$$

```

```

DELIMITER ;

```

```

-----
-- FASE 3: DML - CARGA DE DADOS IDEMPOTENTE
-----

```

```

-- 1. Inserir Pessoas (Usando INSERT IGNORE para evitar duplicidade)
INSERT IGNORE INTO tb_pessoas (pk_pessoa_id, nome, sobrenome, cpf, data_nascimento) VALUES
(1, 'Carlos', 'Bezerra', '111.111.111-11', '1980-01-10'),
(2, 'Ana', 'Silva', '222.222.222-22', '1995-03-12'),
(3, 'José', 'Cauan', '333.333.333-33', '2006-10-11'),
(4, 'Mariana', 'Costa', '666.777.888-99', '1990-04-12');

```

-- 2. Estrutura RH

INSERT IGNORE INTO tb_departamentos (pk_departamento, nome_departamento) VALUES (1, 'Administrativo'), (2, 'Tecnologia');

INSERT IGNORE INTO tb_cargos (pk_cargo, nome_cargo, nivel, salario_base, carga_horaria_semanal) VALUES (1, 'Diretor Geral', 'direcao', 12000.00, 40), (2, 'Analista de TI', 'tecnico', 4200.00, 40);

-- 3. Funcionários

INSERT IGNORE INTO tb_funcionarios (pk_pessoa_id, registro_funcionario, fk_cargo, fk_departamento, regime_trabalho, data_admissao, salario_vigente) VALUES (1, 'FUNC001', 1, 1, 'clt', '2022-01-01', 12000.00);

INSERT IGNORE INTO tb_funcionarios (pk_pessoa_id, registro_funcionario, fk_cargo, fk_departamento, fk_gestor_id, regime_trabalho, data_admissao, salario_vigente) VALUES (4, 'FUNC004', 2, 2, 1, 'clt', '2023-07-01', 4200.00);

-- 4. Acadêmico

INSERT IGNORE INTO tb_alunos (pk_pessoa_id, ra, curso) VALUES (3, '42920949', 'ADS');

INSERT IGNORE INTO tb_cursos (pk_curso_id, nome_curso, carga_horaria) VALUES (1, 'ADS', 2400);

-- 5. Folha de Pagamento

INSERT IGNORE INTO tb_folha_pagamento (fk_funcionario, competencia, salario_bruto, desconto_inss) VALUES (4, '2026-04-01', 4200.00, 462.00);

-- FASE 4: VALIDAÇÃO DE INTEGRIDADE E CONTAGEM

-- Relatório de Contagem para Prova de Idempotência

SELECT 'tb_pessoas' AS tabela, COUNT(*) AS total FROM tb_pessoas
UNION ALL
SELECT 'tb_departamentos', COUNT(*) FROM tb_departamentos
UNION ALL
SELECT 'tb_cargos', COUNT(*) FROM tb_cargos
UNION ALL
SELECT 'tb_funcionarios', COUNT(*) FROM tb_funcionarios
UNION ALL
SELECT 'tb_alunos', COUNT(*) FROM tb_alunos
UNION ALL
SELECT 'tb_cursos', COUNT(*) FROM tb_cursos
UNION ALL
SELECT 'tb_folha_pagamento', COUNT(*) FROM tb_folha_pagamento;

-- Relatório Integrado de Validação

SELECT
 p.nome,
 f.registro_funcionario,
 c.nome_cargo,
 d.nome_departamento,
 f.salario_vigente
FROM tb_funcionarios f
JOIN tb_pessoas p ON f.pk_pessoa_id = p.pk_pessoa_id
JOIN tb_cargos c ON f.fk_cargo = c.pk_cargo
JOIN tb_departamentos d ON f.fk_departamento = d.pk_departamento;

-Script Do Banco

-- Script de Criação do Banco de Dados: Sistema de Gestão Escolar

-- Cria o banco de dados caso ele ainda não exista no servidor

CREATE DATABASE IF NOT EXISTS gestao_escolar;

-- Define o banco de dados 'gestao_escolar' como o banco ativo para os próximos comandos

USE gestao_escolar;

-- 1. Tabela de Cursos: Armazena os cursos oferecidos pela instituição

CREATE TABLE IF NOT EXISTS CURSOS (

cod_curso INT AUTO_INCREMENT PRIMARY KEY, -- Identificador único do curso (Chave Primária), gerado automaticamente

nome_curso VARCHAR(100) NOT NULL UNIQUE, -- Nome do curso, não pode ser nulo e deve ser único no sistema

carga_horaria INT NOT NULL -- Carga horária total do curso, campo obrigatório

) ENGINE=InnoDB; -- Define o motor de armazenamento InnoDB para suportar chaves estrangeiras e transações

-- 2. Tabela de Disciplinas: Armazena as matérias que podem ser vinculadas aos cursos

CREATE TABLE IF NOT EXISTS DISCIPLINAS (

cod_disciplina INT AUTO_INCREMENT PRIMARY KEY, -- Identificador único da disciplina (Chave Primária)

nome_disciplina VARCHAR(100) NOT NULL UNIQUE -- Nome da disciplina, obrigatório e único

) ENGINE=InnoDB;

-- 3. Tabela de Turmas: Agrupa alunos em um determinado curso e ano

CREATE TABLE IF NOT EXISTS TURMAS (

cod_turma INT AUTO_INCREMENT PRIMARY KEY, -- Identificador único da turma (Chave Primária)

cod_curso INT NOT NULL, -- Referência ao curso ao qual a turma pertence (Chave Estrangeira)

nome_turma VARCHAR(50) NOT NULL, -- Nome identificador da turma (ex: Turma A)

ano YEAR NOT NULL, -- Ano letivo da turma (tipo YEAR otimizado para datas de ano)

-- Define a relação: se o curso for excluído, as turmas vinculadas também serão (ON DELETE CASCADE)

CONSTRAINT fk_turma_curso FOREIGN KEY (cod_curso) REFERENCES CURSOS(cod_curso) ON DELETE

CASCADE

) ENGINE=InnoDB;

-- 4. Tabela de Alunos: Cadastro geral de estudantes

CREATE TABLE IF NOT EXISTS ALUNOS (

cod_aluno INT AUTO_INCREMENT PRIMARY KEY, -- Identificador único do aluno (Chave Primária)

nome_aluno VARCHAR(100) NOT NULL, -- Nome completo do aluno

data_nascimento DATE NOT NULL -- Data de nascimento para controle de idade e documentos

) ENGINE=InnoDB;

-- 5. Tabela de Matrículas: Registra o vínculo oficial de um aluno com uma turma específica

CREATE TABLE IF NOT EXISTS MATRICULAS (

cod_matricula INT AUTO_INCREMENT PRIMARY KEY, -- Identificador único da matrícula (Chave Primária)

cod_aluno INT NOT NULL, -- Referência ao aluno matriculado (Chave Estrangeira)

cod_turma INT NOT NULL, -- Referência à turma onde o aluno estudará (Chave Estrangeira)

data_matricula DATE NOT NULL, -- Data em que a matrícula foi realizada

-- Garante que o aluno exista na tabela ALUNOS

CONSTRAINT fk_matricula_aluno FOREIGN KEY (cod_aluno) REFERENCES ALUNOS(cod_aluno) ON DELETE

CASCADE,

-- Garante que a turma exista na tabela TURMAS

CONSTRAINT fk_matricula_turma FOREIGN KEY (cod_turma) REFERENCES TURMAS(cod_turma) ON DELETE

CASCADE

) ENGINE=InnoDB;

-- 6. Tabela de Junção Turma-Disciplina: Define quais disciplinas são ministradas em quais turmas

CREATE TABLE IF NOT EXISTS TURMA_DISCIPLINA (

cod_turma_disciplina INT AUTO_INCREMENT PRIMARY KEY, -- Identificador único deste vínculo (Chave Primária)

cod_turma INT NOT NULL, -- Referência à turma (Chave Estrangeira)

cod_disciplina INT NOT NULL, -- Referência à disciplina (Chave Estrangeira)

-- Relaciona com a tabela de turmas

CONSTRAINT fk_td_turma FOREIGN KEY (cod_turma) REFERENCES TURMAS(cod_turma) ON DELETE CASCADE,

-- Relaciona com a tabela de disciplinas

```
    CONSTRAINT fk_td_disciplina FOREIGN KEY (cod_disciplina) REFERENCES DISCIPLINAS(cod_disciplina) ON  
DELETE CASCADE  
) ENGINE=InnoDB;
```

-- 7. Tabela de Notas: Registra o desempenho acadêmico dos alunos por disciplina

```
CREATE TABLE IF NOT EXISTS NOTAS (  
    cod_nota INT AUTO_INCREMENT PRIMARY KEY, -- Identificador único da nota (Chave Primária)  
    cod_matricula INT NOT NULL,             -- Referência à matrícula do aluno (Chave Estrangeira)  
    cod_turma_disciplina INT NOT NULL,      -- Referência à disciplina específica daquela turma (Chave Estrangeira)  
    data_avaliacao DATE NOT NULL,          -- Data em que a prova ou trabalho foi realizado  
    -- Valor da nota com 2 casas decimais, validando se está entre 0 e 10  
    valor_nota DECIMAL(4,2) NOT NULL CHECK (valor_nota BETWEEN 0 AND 10),  
    -- Vincula a nota ao aluno matriculado  
    CONSTRAINT fk_nota_matricula FOREIGN KEY (cod_matricula) REFERENCES MATRICULAS(cod_matricula) ON  
DELETE CASCADE,  
    -- Vincula a nota à disciplina correta da turma  
    CONSTRAINT fk_nota_td FOREIGN KEY (cod_turma_disciplina) REFERENCES  
TURMA_DISCIPLINA(cod_turma_disciplina) ON DELETE CASCADE  
) ENGINE=InnoDB;
```

-- 8. Tabela de Frequência: Registra a presença dos alunos nas aulas

```
CREATE TABLE IF NOT EXISTS FREQUENCIA (  
    cod_frequencia INT AUTO_INCREMENT PRIMARY KEY, -- Identificador único do registro de frequência (Chave  
Primária)  
    cod_matricula INT NOT NULL,             -- Referência à matrícula do aluno (Chave Estrangeira)  
    cod_turma_disciplina INT NOT NULL,      -- Referência à disciplina da aula (Chave Estrangeira)  
    data_aula DATE NOT NULL,               -- Data da aula realizada  
    presenca BOOLEAN NOT NULL DEFAULT TRUE, -- Indica presença (1/TRUE) ou falta (0/FALSE), padrão é  
presente  
    -- Vincula a frequência ao aluno  
    CONSTRAINT fk_freq_matricula FOREIGN KEY (cod_matricula) REFERENCES MATRICULAS(cod_matricula) ON  
DELETE CASCADE,  
    -- Vincula a frequência à disciplina da turma  
    CONSTRAINT fk_freq_td FOREIGN KEY (cod_turma_disciplina) REFERENCES  
TURMA_DISCIPLINA(cod_turma_disciplina) ON DELETE CASCADE  
) ENGINE=InnoDB;
```


- Erros Corrigidos Da Primeira Entrega

1. Tabela `CURSOS`

- **PK como CHAR(50):** Usar o nome do curso como Chave Primária (PK) não é recomendado. Se o nome do curso mudar, todas as referências em outras tabelas precisarão ser atualizadas.
- **Recomendação:** Criar um `id\_curso` INT AUTO_INCREMENT`.

2. Tabela `TURMAS`

- **PK Duplicada/Confusa:** Você definiu `nome\_curso` como PK, mas turmas diferentes podem pertencer ao mesmo curso.
- **Relacionamento:** Falta uma FK para `CURSOS`.
- **Recomendação:** `id\_turma` INT AUTO_INCREMENT` como PK e `id_curso` INT` como FK.

3. Tabela `NOTAS`

- **Atributos Estranhos:** A tabela contém `presenca`, que geralmente pertence à tabela de `FREQUENCIA`.
- **Relacionamento:** `matricula\_id` está como FK, mas notas geralmente são atribuídas a uma disciplina específica dentro de uma matrícula.
- **Recomendação:** Mover `presenca` para `FREQUENCIA` e garantir que a nota esteja ligada à disciplina.

4. Tabela `TURMA\_DISCIPLINA`

- **Inconsistência de Tipos:** `id` é `BIGINT`, mas as tabelas de origem (Turmas/Disciplinas) não foram definidas com esse tipo.
- **Falta Tabela de Disciplinas:** Você mencionou `disciplina\_id`, mas não forneceu a tabela `DISCIPLINAS`.

5. Tabela `MATRICULA`

- **Confusão de IDs:** Tem `id` e `matricula\_id`. Geralmente, o aluno tem uma matrícula, e a "Matrícula" no contexto de banco de dados é o vínculo do aluno com a turma.
- **Falta Tabela de Alunos:** Para ter uma matrícula, precisamos de um `ALUNO`.

6. Tabela `FREQUENCIA`

- **Duplicação:** Existe `presenca` em `NOTAS` e `Presença` em `FREQUENCIA`.
- **Recomendação:** Centralizar em `FREQUENCIA`.

Proposta de Relacionamentos (ER)

1. **CURSOS** (1:N) **TURMAS**
2. **DISCIPLINAS** (N:N via **TURMA_DISCIPLINA**) **TURMAS**
3. **ALUNOS** (1:N) **MATRICULAS**
4. **TURMAS** (1:N) **MATRICULAS**
5. **MATRICULAS** (1:N) **NOTAS**
6. **MATRICULAS** (1:N) **FREQUENCIA**